

Prompt Engineering for Requirements Engineering: A Literature Review and Roadmap

Kaicheng Huang*, Fanyu Wang*, Yutan Huang* Chetan Arora*

*Faculty of Information Technology, Monash University, Melbourne, Australia

Email: chetan.arora@monash.edu

Abstract—Large-language-model advances have spawned a burst of prompt-engineering (PE) techniques that could streamline many Requirements-Engineering (RE) tasks, yet the field still lacks clear guidance on how to apply them. We present the first roadmap-oriented systematic literature review of Prompt Engineering for RE (PE4RE). Following Kitchenham’s and Petersen’s secondary-study protocol, we searched six digital libraries, screened 867 records, and analyzed 35 primary studies. To bring order to a fragmented landscape, we propose a hybrid taxonomy that links technique-oriented patterns (e.g., few-shot, Chain-of-Thought) to task-oriented RE roles (elicitation, validation, traceability). Two research questions—with five sub-questions—map the tasks addressed, LLM families used, and prompt types adopted, and expose current limitations and research gaps. Finally, we outline a step-by-step roadmap showing how today’s ad-hoc PE prototypes can evolve into reproducible, practitioner-friendly workflows.

I. INTRODUCTION

Requirements Engineering (RE) is fundamental to the success of every software project [1]. With the rapid advancements in large language models (LLMs) and other natural language processing (NLP) technologies, NL specifications have become increasingly common. These specifications can be found in user stories within agile backlogs, regulatory clauses in safety-critical domains, and free-form stakeholder comments stored in issue trackers. Their growing prevalence encourages using NLP techniques in various RE tasks, including elicitation, classification, conflict detection, traceability-link recovery, and even automated test generation [2].

Due to the complexity of state-of-the-art large language models (LLMs), which contain hundreds of billions of parameters, most researchers in requirements engineering (RE) access these models primarily through hosted inference APIs rather than by fine-tuning the entire models. This shift in access changes the focus from model architecture to prompt engineering (PE)—the art of crafting input queries that guide an LLM toward producing accurate, consistent, and context-appropriate outputs [3].

PE techniques include methods such as few-shot priming [4], chain-of-thought reasoning [5], retrieval-augmented generation [6], constraint injection, and multi-role dialogue orchestration. These techniques have become essential for applying LLM capabilities within the RE community. However, there is still a lack of comprehensive understanding of the full range of PE techniques and their trade-offs in the RE community. As a result, researchers often treat prompting as an ad hoc implementation detail rather than a deliberate design choice with significant effects on outcomes.

Consequently, researchers might (i) choose popular prompting patterns, such as “Act as a requirements analyst,” without verifying whether the framing aligns with relevant downstream artifacts (e.g., traceability matrices, compliance reports); (ii) develop elaborate task-specific templates while neglecting to conduct ablation studies, which could reveal which elements of the prompt—such as examples, reasoning cues, or structural constraints—are responsible for any observed improvements; and (iii) report isolated success metrics that obscure the practical limitations of their approach. Without a unified framework mapping how PE strategies connect with RE tasks, the field struggles to determine which techniques are the most effective, where they fall short, or what realistic benefits practitioners can anticipate.

To overcome the limitations, we proposed a guideline paper that systematically analyzes existing studies and provides a roadmap for future developers in this field. Our contributions are listed as follows:

- Following Kitchenham et al. [7]’s guidelines for secondary studies and Petersen et al. [8]’s mapping methodology, we searched six major digital libraries, filtered 867 records down to 35 primary studies, and extracted detailed data about their LLM choice, prompt design, RE task, and more.
- We map existing studies into our proposed PE classification from both the feature and task perspectives and systematically discuss the advantages and disadvantages in the context of RE tasks.
- A comprehensive guideline is proposed for future researchers with the potential limitations and opportunities.

Structure. Section II describes the background of Requirements Engineering (RE) and Prompt Engineering (PE). Section III specifies the methodology of our literature review. Section IV-A gives the categorization of PE techniques in the context of RE domain. Section V presents our analysis results and the roadmap discussion. Section VII concludes our study.

II. BACKGROUND

A. Requirements Engineering

Requirements Engineering (RE) is the systematic discipline devoted to eliciting, analysing, documenting, validating, and managing the requirements that define a software system’s scope and success [9]. By framing the real-world problems a solution must address, RE acts as the conduit between stakeholder needs and technical implementation, negotiating diverse

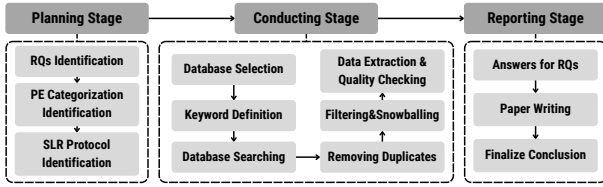


Fig. 1. Procedure of the Study

perspectives to reach a set of mutually acceptable, verifiable requirements [10], [11]. Its cyclical workflow—commonly decomposed into elicitation, analysis, specification, and validation, with ongoing requirements management—ensures that emerging insights and changing contexts are continually fed back into the project [12]. High-quality RE mitigates costly rework and requirements drift by grounding design, testing, and maintenance activities in a shared, traceable understanding of system goals and constraints. As software projects grow in scale and complexity, new techniques such as LLM-driven analysis and automated traceability are reshaping traditional RE practices, offering opportunities to accelerate information processing while reinforcing the field’s enduring mandate: delivering the right software for the right users.

B. Prompt Engineering

Prompt engineering (PE) is the emerging discipline that treats the prompt—the textual query given to a large language model (LLM)—as the primary lever for controlling model behaviour. Formally, PE is the design of a prompting function that maximises downstream task performance without altering model parameters [13]. A prompt typically weaves together (i) instructions that stipulate the task, (ii) context that grounds the model in domain-specific background or examples, and (iii) the input to be processed. Because modern LLMs are too large to fine-tune routinely, practitioners access them through hosted APIs; PE therefore offers a cost-effective alternative to retraining, enabling rapid adaptation to new domains [14]. Crafting effective prompts blends analytic rigour with iterative experimentation: designers test, debug, and refine phrasing, ordering, and example selection until the model yields outputs that are precise, verifiable, and aligned with domain constraints [15], [16]. The practice has been likened to a new form of programming—complete with its own design patterns (e.g., few-shot priming, chain-of-thought, retrieval-augmented prompts) and taxonomies that distinguish open- versus closed-ended, instructive versus conversational, and simple versus composite structures [17]. As LLMs demonstrate state-of-the-art performance across fields from healthcare to software engineering, proficiency in PE has become a sought-after skill and a prerequisite for deploying generative models in safety- and quality-critical settings.

III. SYSTEMATIC LITERATURE REVIEW OF PE IN RE

In this section, we outline our systematic review methodology, including the search strategy and data extraction procedures. Our approach follows the guidelines proposed by Kitchenham et al. [7] for conducting systematic literature

TABLE I
SEARCH STRINGS

Groups	Search Strings
PE Group	“prompt engineering” OR “prompt design*” OR “prompt creat*” OR “prompt construct*” OR “prompt pattern*” OR “prompt-based” OR “PE”
RE Group	“requirement* engineering” OR “requirements analy*” OR “requirements specification*” OR “software requirement*” OR “requirements elicit*” OR “requirements valid*” OR “requirements manage*” OR “SE” OR “RE”
LLMs Group	“LLM” OR “Large Language Model*” OR “Language Model*” OR “LM*” OR “PLM*” OR “Pre-trained” OR “Pre-training” OR “Transformer” OR “generative AI” OR “GenAI” OR “generative artificial intelligence”

reviews (SLRs), and Petersen et al. [8] for conducting preliminary literature reviews, particularly in the context of our roadmap paper. There are three stages in our SLR, including (1) the *planning* stage involves identifying research questions, particularly from a roadmap perspective, and outlining a detailed research plan for the study; (2) the *conducting* stage involved analyzing and synthesizing existing primary studies to answer potential questions; (3) the *reporting* stage involved analyzing, concluding, and discussing the PE4RE studies in context of advanced PE researches and identifying the challenges, limitations and future directions in this field. The overview of this work is presented in Fig. 1.

A. Research Questions in Roadmap

1) *RQ1. What are the current state-of-the-art studies of PE4RE?*: Numerous studies have utilized PE methods to enhance the RE process. In this RQ, we will primarily focus on the technical aspects of prompting methods, specifically discussing (1) the large language models (LLMs) used and (2) the categorization of the prompting methods.

2) *RQ2. What is the future of PE4RE in view of the advanced PE technologies?*: From PE (NLP domain) to RE (SE domain), we acknowledge the research and technical gap between these two different fields. In RQ2, we compare the current PE4RE research with the advanced PE studies to identify limitations or challenges, then we discuss the potential solutions and outline a roadmap of PE4RE research in the context of our proposed categorization of PE studies.

B. Searching Procedure

1) *Database Selection*: We selected six online databases for our survey, including IEEE Xplore, ACM Digital Library, Wiley, Scopus, Web of Science, and Science Direct. We acknowledge that there are other studies that have adopted Arxiv as their database. But considering that some of the studies on Arxiv have not been completely reviewed, we didn’t include Arxiv in our database selection.

2) *Searching Strings*: To cover all the alternative expressions, we adopted three groups of searching strings by exhausting all regular combinations of our related topics and using advanced search commands, such as “AND”, “OR”, and “*”, to conduct the literature search.

3) *Selection Criteria*: We present our selection criteria in Table. II. Taking into account some similar expressions in the other field, we select the subject of “computer science” to

TABLE II
INCLUSION AND EXCLUSION CRITERIA

ID	Description
Selection Criteria	
S01	Papers written in English.
S02	Papers in the subject of “Computer Science”.
S03	Papers published from 2018 to the present.
S04	Papers with accessible full text.
Inclusion Criteria	
I01	Focuses on at least one RE task, i.e., requirements elicitation, analysis, specification, validation, or management in SE.
I02	Use prompt engineering strategies in the methodology – even if the paper does not explicitly use the term “prompt engineering,” these techniques should be evident in how the solution is implemented.
Exclusion Criteria	
E01	Studies that only mentioned prompt engineering but no details or guidelines.
E02	Papers with fewer than 5 pages (1-4 pages).
E03	Vision papers and grey literature (unpublished work), books (chapters), posters, discussions, keynotes, magazine articles, and comparison papers.

TABLE III
DETAILS OF SEARCHING RESULTS

Database	Searching	Remove Duplicate	Final
ACM	191	174	4
IEEE	134	134	14
Wiley	2	2	0
WOS	246	131	5
ScienceDirect	36	36	3
Scopus	258	61	5
Total	867	538	31 + 4 (Snowballing)

avoid returning some irrelevant papers. As for S03, due to the first release of “LLM” or “pretrained-LM” is in 2018, we applied this criteria to ensure the correct coverage.

4) *Inclusion and Exclusion Criteria*: After we conducted the search on the selected databases, we designed a set of inclusion and exclusion criteria for further screening the irrelevant papers, which is exhibited in Table II. In I01, we aim to exclude the papers without investigating requirements engineering. I02 requires the papers to propose at least one type of prompting method in their methodology.

5) *Searching Results*: The search process was conducted in April 2025, and therefore, the search results reflect studies available up to that date. We present the details of paper counts among different databases in Table III. The total number of papers after filtering and snowballing is 35 (see Appendix A).

IV. CLASSIFICATION OF PROMPT ENGINEERING IN RE

A. Existing Classification in Prompt Engineering

Prompt engineering, based on different perspectives, is broadly categorized into different taxonomies, which are mainly classified into 1) task-oriented, grouping PE techniques by their application scenario, such as retrieval-augmented generation (RAG), multimodal generation, or code generation, and 2) technique-oriented, focusing on individual methods such as few-shot learning, zero-shot learning, or Chain-of-Thought. However, presenting these categories at the same hierarchical level can confuse readers: for example, few-shot learning is often used within code generation, yet it appears alongside “code generation” rather than as a subcategory. In response, Sahoo et al. [18] analyze prompt engineering across function, technique, and application, and they propose twelve categories: 1) new tasks without extensive training, 2) reasoning and logic, 3) reduce hallucination, 4) user interaction, 5)

fine-tuning and optimization, 6) knowledge-based reasoning and generation, 7) improving consistency and coherence, 8) managing emotions and tone, 9) code generation and execution, 10) optimization and efficiency, 11) understanding user intent, and 12) metacognition and self-reflection. While this taxonomy provides broad coverage, it still mixes different levels of abstraction—“reduce hallucination,” for instance, is a technique that overlaps with “knowledge-based reasoning and generation” as methods to prevent misinformation, making it difficult to distinguish technique from application. From this observation, we conclude that the categorization of prompt engineering approaches should follow a clearer logical hierarchy, separating techniques, objectives, and application domains. Fagbohun et al. [19] offer an alternative taxonomy driven by function rather than by individual characteristics or techniques: they define seven functional categories—logical and sequential processing, contextual understanding and memory, specificity and targeting, metacognition and self-reflection, directional and feedback, multimodal and cross-disciplinary, and creative and generative. By emphasizing functional groupings rather than isolated methods like “reduce hallucination,” this framework presents a more standardized, interdisciplinary structure. Nevertheless, its inclusive definitions can also blur boundaries—for example, including code generation under “metacognition and self-reflection” lacks a clear conceptual link, since code generation does not inherently involve self-reflective processes. Therefore, even a function-based taxonomy must ensure that each category genuinely encompasses techniques sharing the same underlying function.

B. PE Categorization

Inspired by existing PE surveys, we conclude their PE taxonomy and propose our PE taxonomy (in Fig. 2) from a hybrid perspective from *task-oriented* approaches to *technique-oriented* approaches, which aims to maintain extensive coverage while ensuring a consistent categorization logic in RE context. We also cover examples based on a common scenario for each category. *In this scenario, you’re designing a smart home thermostat. You should meet the following requirements:*

- *measure ambient temperature,*
- *display it on a screen,*
- *let users set a target temperature,*
- *adjust the HVAC system to reach that target,*
- *log the temperature once per minute, and*
- *send an alert if temperature deviates > 5 degree from target for > 10 min.*

Creative Generation. The motivation behind creative generation prompting is to push LLMs beyond factual or routine responses, encouraging novelty and innovation in output. In RE, stakeholders are routinely asked to imagine a future that does not yet exist. Creative generation can help produce engaging requirement artifacts, alternative requirement formulations, elicit out-of-the-box feature ideas, and innovative solutions to requirement conflicts. In the example below, we show how creative generation can be used in RE. The creative burst can be followed by an evaluation step to establish the feasibility

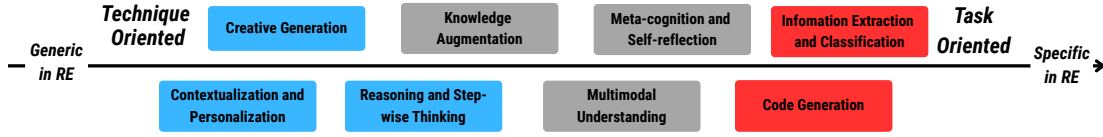


Fig. 2. PE Categorization: Blue are technique-oriented categories, Grey enable both techniques and task-specific, and Red are task-oriented categories.

and business value, then translate promising concepts into precise requirements, e.g., “System shall update LED strip intensity within 500 ms of a detected ± 0.5 °C change.”

```
<USER>:
Inspire our design team! Brainstorm an imaginative new feature for a next-gen smart thermostat. Present it as a short, one-sentence bullet point.

<LLM>:
1. Aurora-Glow Mode paints the room with gentle colour waves that ebb and flow with temperature shifts, turning climate control into ambient art.
2. Whisper-Forecast murmurs tomorrow's weather in a soft chime each dawn, nudging the thermostat to pre-warm or pre-cool before you even wake.
3. Harmony-Pulse Sync listens to household music and subtly adjusts airflow rhythm so the room “breathes” in time with your favourite playlist.
```

Listing 1. Creative Generation

Contextualization and Personalization. This category encompasses influential prompting techniques, including one-shot, few-shot, and in-context learning. These techniques are also useful and commonly adopted in RE [20]. RE involves multiple stakeholders with different backgrounds, concerns, and technical expertise. Each stakeholder group needs requirements presented in their context and terminology. By anchoring generation in this tailor-made context, these prompts markedly improve consistency, reduce redundant explanations, and allow the assistant to adopt a stable voice that adapts to each user’s knowledge level and style, thereby boosting engagement and trust in long-running dialogues. Typical techniques include in-context prompts [21], multi-persona or role prompts, emotion prompting [22], and conversational summarisation buffers, which prepend recent turns, user preferences, or persona instructions before each generation step.

```
<USER>: Example (two-audience list)
Q: List two key requirements for a humidity sensor for two stakeholder perspectives: Homeowner (1) and HVAC engineer (2)
1. - Warns you when air is too damp or dry
   - Shows humidity on your phone
2. -  $\pm 2$  % RH accuracy at 25 °C
   - Publishes readings every 60 s via MQTT/TLS
Task: List two key requirements for a smart thermostat in the same way.

<LLM>:
1. - Keeps your rooms at the temperature you set
   - Lets you change settings from a mobile app
2. - Maintains  $\pm 0.5$  °C via PID control at 1 Hz
   - Logs data and supports OTA firmware updates
```

Listing 2. Contextualization and Personalization

Reasoning and Step-wise Thinking. RE sub-tasks often involve complex logical reasoning to derive functional requirements from business needs, trace dependencies between requirements, and validate requirement consistency. LLMs can struggle with multi-step requirement analysis tasks, preferring shortcuts over systematic RE methodologies. To address this issue, Reasoning and Stepwise Thinking PE researchers break the complex reasoning tasks into stepwise reasoning paths to let LLM think through a logical reasoning chain. The

typical approach under this category is CoT series [5], [23], [24]. These CoT-styled prompt templates are widely employed in various tasks, including Tree-of-Thoughts [25], Graph-of-Thoughts [26], and so on. These techniques can be widely employed in different stages of RE, such as requirements decomposition & validation, dependency analysis, etc.

```
<USER>:
Please identify all functional and non-functional requirements in the thermostat scenario. First, list every candidate requirement. Then, categorize each as FR or NFR. Finally, explaining your reasoning step by step.

<LLM>:
Step 1: List candidate requirements. Measure ambient temperature – Display temperature on screen – Let user set target temperature – Adjust HVAC to target – Log temperature every minute – Send alert if deviation > 5 degree or > 10 min.
Step 2: Categorize. Measure ambient temperature – FR (it’s a core function) – Log temperature every minute – NFR (performance: frequency) – etc.
```

Listing 3. Example: Reasoning and Step-wise Thinking

Knowledge Augmentation. RE activities benefit significantly from external knowledge sources, including industry standards, regulatory requirements, best practices, and domain expertise that may not be captured in LLM training data. Knowledge-augmentation prompts. Therefore, inject trustworthy external information—most often via Retrieval-Augmented Generation (RAG) [6] pipelines or tool-use schemas—to ground the model’s reasoning on up-to-date sources. Representative techniques include RAG, ReAct [27] and Chain-of-Knowledge prompting [28], which prepend retrieved passages, interleave “thought-action-observation” traces, or have the model enumerate supporting facts step-by-step. By explicitly supplying evidence within the prompt, these methods reduce hallucinations, increase factual accuracy, and make answers verifiable, which can be applied in some RE stages that require context information, such as integrating regulatory requirements and domain knowledge. The knowledge augmentation prompt must be based on some external knowledge resource. By performing the information retrieval method, the related context can be used for prompt construction.

```
<CONTEXT>:
Assume the extracted content from the external knowledge resource (ASHRAE Handbook):
Best practice: log temperature data at 5 min intervals for energy audits; alarms recommended at 4 degree deviation over 15 min.

<USER>:
Using the ASHRAE excerpt above, update the thermostat’s NFRs (logging interval and alert thresholds) and justify each change.

<LLM>:
Logging Interval: Change from 1 min to 5 min. Justification: Aligns with ASHRAE energy audit standards to reduce data storage overhead while still capturing enough resolution.
Alert Threshold: Change from 5 degree over 10 min to 4 degree over 15 min. Justification: Matches industry guidelines for fault detection without triggering nuisance alarms during normal HVAC cycling.
```

Listing 4. Example: Knowledge Augmentation

Multimodal Understanding. RE widely accepts various types of specifications, including UI mockups, system diagrams,

process flows, and prototype demonstrations that need to be analyzed alongside textual requirements. Prevailing frameworks either (i) pair specialized vision or audio encoders with a language decoder via a lightweight bridging module, as in BLIP-2 [29], or (ii) train a single Transformer end-to-end on interleaved modality tokens, exemplified by Flamingo [30], Kosmos-1 [31] & Kosmos-2 [32], and GPT-4V [33]. While offering greater robustness and context fidelity, multimodal models incur higher computational cost, risk modality imbalance, and can compound data-borne biases. Overall, even multi-modal prompting techniques are not commonly adopted in the RE community; however, they can still be applied in most stages, particularly for requirements elicitation from domain documents and meta-model-driven software design.

```
<FIGURE>:
(An image will be attached in the prompt, which is a screenshot of the thermostat's
touchscreen UI.)
<USER>:
Here's the current UI mockup. Identify three usability issues that might confuse users,
then suggest a quick redesign note for each.
<LLM>:
Tiny Font for Temperature: Difficult to read at a glance. Redesign: Enlarge digits to at
least 24 px and use high-contrast colors.
Unlabeled Buttons: '+' and '-' icons without context. Redesign: Add tooltip or
label ('Set Temp +/-').
No Status Indicator for Logging: Users can't tell if logging is active. Redesign:
Include a small 'Logging' LED icon in the corner.
```

Listing 5. Example: Multimodal Understanding

Meta-cognition and Self-reflection. While LLMs respond to input queries, models may commit logical slips or confidently output flawed facts. Self-reflective prompting forces them to critique and, if needed, revise those first drafts, which can assure the quality of the response. Quality is also critical in RE, as requirement defects are expensive to fix later in development. Self-reflection helps ensure requirement completeness, consistency, and quality through systematic review processes. Core approaches—such as Reflexion [34], Meta-Cognition [35], Self-Explanation [36] and Step-Back prompting [37], either generate multiple independent solutions and elect the majority answer, or insert an explicit loop from critique to revise where the model reviews its own reasoning trace for gaps, bias or factual errors before finalising. The most typical application scenario can be requirements review. The key feature of this category is that the prompt will utilise multiple rounds of instruction to allow LLMs to reconsider and double-check their responses.

```
<PREVIOUS CONVERSATION>:
(First Round) First, we ask it to extract FRs/NFRs (as in the example of Reasoning
and Step-wise Thinking.)
<USER>:
You just pulled out six requirements. Are you confident none are missing? If any gaps
exist, critique and add them.
<LLM>:
On reflection, I omitted the security requirement. FR: Authenticate user before
allowing target-temp changes. Rationale: Prevents unauthorized HVAC control.
Now there are seven requirements in total.
```

Listing 6. Example: Meta-cognition and Self-reflection

Information Extraction and Classification. RE practitioners sometimes need to extract, categorize, and structure requirements from various sources, including stakeholder interviews, existing documentation, legacy systems, and regulatory documents. The information extraction and classification refer to

extracting or analyzing the external resources, which always cooperate with contextualization and personalization prompting methods in the RAG [6] framework. The application of Information Extraction and Classification in the RE process includes requirements classification and priority classification.

```
<USER>:
From the scenario text, extract every requirement into CSV with columns ID, Type,
Text, where Type is FR or NFR.
<LLM>:
*R1, FR, Measure ambient temperature
*R2, FR, Display temperature on screen
```

Listing 7. Example: Information Extraction and Classification

Code Generation. Code here refers to a concrete SE artifact, encompassing system implementations, software design, and testing. In many RE projects we need a quick, runnable artefact to check that a requirement has been understood correctly, before investing weeks in full implementation. Code generation lets analysts move straight from text to artifacts of interest, closing the gap between “specify” and “validate”. Code demands strict syntax and logical correctness, so prompts must be specially designed. Along with the other prompting techniques, the code generation prompt has two special steps, including (1) context provisioning, which gives the description to LLMs to specify the objectives, and (2) structured specification, which regulates LLMs for the output format, including the scale and the programming language. For the example below, with a single prompt below we now have an executable specification that stakeholders can run, developers can extend, and auditors can trace—illustrating the practical value of code-generation prompts in RE.

```
<CONTEXT>
R1, FR, "System shall log temperature every 60 s"
R2, NFR, "Temperature accuracy shall be within 0.5 C"

<USER>
Write a PyTest file that:
1. Creates one test per requirement.
2. Tags each test with @pytest.mark.req("<ReqID>").
3. Fails if logging interval ≠ 60 s or accuracy drift > 0.5 C.

<LLM>
# test_thermostat_reqs.py
import thermostat, pytest

@pytest.mark.req("R1")
def test_log_interval():
    assert thermostat.LOG_INTERVAL == 60

@pytest.mark.req("R2")
def test_temp_accuracy():
    diff = abs(thermostat.read_temp() - thermostat.true_temp())
    assert diff <= 0.5
```

Listing 8. Code Generation

Our categorization is designed to be mutually exclusive yet collectively exhaustive. By anchoring each category in a distinct primary function, this taxonomy retains the strengths of earlier work while offering sharper conceptual boundaries and providing a clearer map from real-world prompting goals to the design strategies best suited to achieve them.

V. RESULTS AND DISCUSSION

A. RQ1 Results

RQ1-1. What is the end-goal of the selected studies? In

TABLE IV
OVERVIEW OF END-GOALS IN SELECTED STUDIES

PID	Target Task	Task Categorization
P1	Goal Model Extraction	Model Gen.
P2	Software Conflict Detection	Software Analysis
P3	Software Safety Analysis	Safety Case Gen. and Analysis
P4	Use Case Classification	Req. Analysis and Gen.
P5	Automated Test Gen.	Test Gen.
P6	Software Development ChatBot	SDLC Assistant
P7	Req. driven Testing	Test Gen.
P8	Eval. of Req. Analysis Against Human	Req. Analysis and Gen.
P9	Assurance Case Defeater Identification	Defeater Identification
P10	Enhancement of Code Gen. Capability	Code Analysis and Gen.
P11	Defect Correction in Req.	Defect Correction
P12	Domain Model Extraction	Model Gen.
P13	Enhancement of Use Case	Req. Analysis and Gen.
P14	Enhancement of Use Case Gen.	Req. Analysis and Gen.
P15	Domain Model Extraction	Model Gen.
P16	Multi-agent Sys. for Elicitation/Analysis	Req. Analysis and Gen.
P17	RE-driven Java Code Gen.	Code Analysis and Gen.
P18	Generating Req. Elicitation Interview Scripts	Req. Analysis and Gen.
P19	Transforming Software Req. into User Stories	Req. Analysis and Gen.
P20	Sensor Input Setup Assistant	Software Analysis
P21	Software Development Assistant for Communication System	SDLC Assistant
P22	RE Assistant for Digital Twins	Req. Analysis and Gen.
P23	Evolving Req. Elicitation from App Reviews	Req. Analysis and Gen.
P24	Natural Language Inference (NLI) in Req. Engineering Tasks	Req. Analysis and Gen.
P25	LLMs for Software Engineering Education and Training	Software Analysis
P26	Req. driven UML Sequence Diagrams	Model Gen.
P27	Automatic Gen. of Formal Req. Specifications	Req. Analysis and Gen.
P28	GPT-4 for Creating Goal Models	Model Gen.
P29	Automatic Safety Case Gen.	Safety Case Gen. and Analysis
P30	Req. Modeling Aided by ChatGPT	Req. Analysis and Gen.
P31	Req. Satisfiability Analysis	Req. Analysis and Gen.
P32	Req.-driven Slicing of Simulink Models	Model Gen.
P33	Acceptance Case Gen.	Test Gen.
P34	Req. Coverage Reviewing	Req. Analysis and Gen.
P35	Automated User Story Gen.	Req. Analysis and Gen.

the selected studies, we identified that there are various types of end-goals and tasks included, i.e., P1 and P26 are both focused on model generation, P10 and P17 are both for code generation. Thus, we present a classification of all the end goals in Table. IV and V.

- Requirements Analysis and Generation is the most common task in the selected papers (15 studies). These studies directly employ prompt engineering to optimize the requirements engineering process, including requirements optimization (P13, P14, P22), use case classification (P4), evaluation of requirements analysis (P8, P24, P31, P34), requirements generation (P16, P18, P19, P23, P27, P30, P35). Considering the scope is PE4RE, it is reasonable there are studies directly related to requirements engineering. Besides, with the adoption of natural language requirements, such as user stories and use cases, prompt engineering can enhance the processing capabilities of NL by leveraging advancements of LLMs.
- Model Generation is the second most popular goal in the selected papers (6 studies). While meta-model requirements specification is widely adopted in requirements engineering process. However, the formulation of meta-model-based requirements is always a challenge in RE

TABLE V
CATEGORIES OF PROMPTING METHODS

Task Categories	PIDs	Num.
Model Generation	P1, P12, P15, P26, P28, P32	6
Software Analysis	P2, P20, P21, P25	4
Safety Ana. & Gen.	P3, P29	2
Req Ana. & Gen.	P4, P8, P13, P14, P16, P18, P19, P22, P23, P24, P27, P30, P31, P34, P35	15
Code Ana. & Gen.	P10, P17	2
Test Generation	P5, P7, P33	3
SDLC Assistant	P6, P21	2
Defeater Identification	P9	1
Defect Correction	P11	1

TABLE VI
THE NUMBER OF LLMs ADOPTED IN SELECTED STUDIES

LLM Types	PIDs	Num.
GPTs	P1 (GPT4), P3 (GPT4o), P4 (GPT3.5-Turbo), P5 (GPT4), P6 (GPT3.5-Turbo, GPT4), P7 (GPT3.5-Turbo, GPT4), P8 (GPT3.5-Turbo, GPT4), P9 (GPT3.5-Turbo), P10 (GPT3.5-Turbo, GPT4), P11 (GPT3.5, GPT4), P12 (GPT4), P13 (GPT3.5-Turbo), P14 (GPT4-Turbo, GPT3.5-Turbo, GPT3.5-Turbo-instruction), P15 (GPT3.5-Turbo), P16 (GPT3.5, GPT4), P17 (GPT3.5-Turbo), P18 (GPT3.5), P19 (GPT3.5), P20 (GPT4o), P21 (GPT3.5), P22 (GPT3.5), P23 (GPT4), P24 (GPT3.5, GPT4), P25 (GPT3.5), P26 (GPT3.5), P27 (GPT3.5), P28 (GPT4), P29 (GPT4), P30 (GPT3.5-Turbo, GPT4), P31 (GPT3.5-Turbo, GPT4), P32 (GPT3.5-Turbo), P33 (GPT3.5-Turbo), P34 (GPT3.5, GPT4), P35 (GPT4-Turbo)	34
Llama	P1 (Llama-13B), P4 (Llama2-70B), P5 (Llama2-70B), P10 (CodeLlama-34B), P25 (LLaMA-2-70B), P33 (Llama-7B & 13B)	6
Other	P1 (Cohere-Command), P8 (Nous-Hermes-2), P10 (CodeGeeX), P11 (ERNIE-Bot), P14 (Whisper), P18 (Bard), P27 (T5-Large), P33 (text-davinci-003)	8
PLM	P2 (BERT, ALBERT, RoBERTa, XLNet), P11 (BERT), P23 (BERT), P24 (RoBERTa, NoBERT, BERT)	4
Qwen	P3 (Qwen-Max)	1
Gemini	P5 (Gemini-Pro)	1
Mistral	P5 (Mistral-7B), P25 (Mixtral-8x7B)	2
Phi	P8 (Phi-3-Medium)	1
Deepseek	P10 (DeepSeek-Coder-V2)	1
Claude	P21 (Claude 3)	1

domain [38], [39]. By the help of the prompt engineering methods, the generalization ability of LLMs is exhausted in model generation task.

- There are four studies related to Software Analysis topic. These studies not only focus on the requirements engineering field, but also use requirements to assist the analysis of software artefacts, such as conflict detection (P2) and sensor input setup (P20), which have broader application scenarios.
- Code Analysis and Generation is also a attractive task in recent research. As a highly structural language, a high-quality prompt template can significantly enhance the code generation ability by minimizing ambiguous input and preventing hallucination.

RQ1-2. Which LLMs are adopted in the selected papers? In this sub-RQ, we present all the applied LLMs in the selected papers. Due to different preferred input style among various LLMs, the selection of LLMs in prompt engineering research can somehow reflect the generalization ability of prompting methods. We present a comprehensive list of adopted LLMs in Table. VI, which suggest that,

- GPT-series is the dominant choice in the selected papers. As the most popular LLM, GPTs provide one of the

friendliest interaction websites and interfaces, stable API inference, and various plugins. The common preference for GPTs reflects their strong generalization ability and their representative roles among all LLMs. Since prompting is typically tied to training or fine-tuning, the prompt engineering method will remain effective even if GPTs do not support gradient visibility.

- Llama is in second place in LLM selection. Different from GPTs, Llama is an open-source LLM that supports customization and fine-tuning.
- Some task-specific LLMs are adopted in the selected paper, i.e., Copilot, CodeLlama, and DeepSeek-Coder, which are known for their leading role in code generation area, the selection of these LLMs aligns with the end-task in RQ1-1, which suggests that prompt designing should widely consider both task-specific and generic LLMs.

RQ1-3. How are PE4RE studies in the context of PE categorization? As we introduced our categorization method in SectionIV-A, we further map the selected studies into our defined categorizations, which are shown in TableVII and VIII. Based on the results, we found that Contextualisation & Personalisation (26 studies) and Reasoning & Step-wise Thinking (20 studies) are the top-2 adopted prompt-engineering methods in the selected studies, followed by Creative Generation (14 studies), Information Extraction & Classification (11 studies), Knowledge Augmentation (8 studies), Meta-cognition & Self-reflection (6 studies), Code Generation (4 studies), Defeater Identification / Defect Correction (1 studies), Multimodal-Understanding (0 studies). As for the co-occurrence patterns between the multiple types of prompt-engineering methods, we have the following conclusions, (1) Contextualisation & Personalisation and Reasoning & Step-wise Thinking are always adopted together (14/20), which suggest that the dominant recipe is role prompt + chain-of-thought, suggesting researchers view rich context as a prerequisite for trustworthy reasoning; (2) Knowledge Augmentation methods always adopted with the other techniques, as 4/8 RAG-style papers also include Reasoning prompts, which indicates that researchers tend to use retrieved snippets as extra context rather than as a distinct prompting paradigm; (3) Meta-cognition & Self-reflection methods are almost always layered on top of Reasoning & Step-wise Thinking, where verifier–reviser or self-consistency loops require step-wise traces to critique, reinforcing the centrality of explicit reasoning in RE prompts.

B. RQ2 Results

RQ2-1: What are the limitations and challenges in current PE4RE studies? In this sub-RQ, we aim to identify the limitations and challenges in our selected studies and the potential influence of these challenges. Across the 35 primary studies we analysed, we identified limitations as,

- 1) *No studies employ non-textual prompting methods, such as Multi-Modal Understanding, to process non-textual input.* We notice that, except for P26 and P30, which produce UML, P32 converts models to text, the other studies

TABLE VII
OVERVIEW OF PROMPTING METHODS

PID	Prompting Methods
P1	Reasoning & Step-wise Thinking (chain-of-thought style 7-step prompts); Contextualization & Personalization (role-prompt “You are a RE engineer...”, domain background supplied)
P2	Info. Extraction & Classification (classifies SRS pairs into Conflict / Duplicate / Neutral)
P3	Knowledge Augmentation (retrieval-augmented Gen. against safety regulations); Meta-cognition & Self-reflection (Researcher & Revisor loop); Contextualization & Personalization (agents given explicit functional-safety roles)
P4	Info. Extraction & Classification
P5	Reasoning & Step-wise Thinking (CoT); Contextualization & Personalization (few-shot examples)
P6	Reasoning & Step-wise Thinking (ReAct with explicit action steps); Contextualization & Personalization (conversation history + module state)
P7	Knowledge Augmentation (RAG pipeline); Contextualization & Personalization (optional exemplar scenario = few-shot)
P8	Contextualization & Personalization (few-shot practice doc + example defects)
P9	Contextualization & Personalization (role-based system prompt)
P10	Reasoning & Step-wise (Structured CoT); Contextualization (role-definition prompts); Meta-cognition & Self-reflection (internal code-review stage); Code Gen.
P11	Contextualization & Personalization (rich upstream cues); Info. Extraction & Classification (defect detection); Knowledge Augmentation (domain-specific keywords); Creative Gen. (rewriting correct sentences)
P12	Info. Extraction & Classification ; Reasoning & Step-wise Thinking (OpenAI and existed prompt patterns); Contextualization & Personalization (guidance examples)
P13	Contextualization & Personalization (persona pattern); Meta-cognition & Self-reflection (feedback loop & self-revision); Creative Gen. (rewriting use-cases)
P14	Creative Gen. ; Contextualization & Personalization
P15	Info. Extraction & Classification ; Reasoning & Step-wise Thinking
P16	Contextualization & Personalization (role prompts); Reasoning & Step-wise Thinking
P17	Code Gen. ; Knowledge Augmentation ; Contextualization & Personalization ; Reasoning & Step-wise Thinking
P18	Reasoning & Step-wise Thinking ; Creative Gen. ; Contextualization & Personalization
P19	Creative Gen. ; Contextualization & Personalization (few-shot examples)
P20	Knowledge Augmentation ; Contextualization & Personalization ; Reasoning & Step-wise Thinking ; Info. Extraction & Classification
P21	Code Gen. ; Reasoning & Step-wise Thinking ; Contextualization & Personalization ; Meta-cognition & Self-reflection (inspector-critique loop)
P22	Creative Gen. (LLM-authored requirements); Reasoning & Step-wise Thinking (iterative prompting to improve completeness & correctness)
P23	Contextualization & Personalization (R-T-O-QA template grounds the task); Knowledge Augmentation (300 k-entry sentiment lexicon injected); Info. Extraction & Classification (aspect-based SA)
P24	Info. Extraction & Classification (entailment); Reasoning & Step-wise Thinking (logic of entail & contradict); Contextualization & Personalization (label verbalisation = rich hypotheses)
P25	Creative Gen. (idea lists, code); Contextualization & Personalization (role-play, tutor personas); Reasoning & Step-wise Thinking (effort-estimation dialogue)
P26	Creative Gen. (diagram synthesis); Reasoning & Step-wise Thinking (LLM must infer interactions)
P27	Creative Gen. (dataset Gen.); Info. Extraction & Classification (AP recognition); Code Gen. (CTL code output)
P28	Creative Gen. (model Gen.); Contextualization & Personalization (injecting domain context)
P29	Creative Gen. ; Contextualization & Personalization ; Info. Extraction & Classification
P30	Info. Extraction & Classification (entity & relation extraction); Creative Gen. (diagram synthesis); Contextualization & Personalization (prompt embeds element definitions)
P31	Reasoning & Step-wise Thinking (chain-of-thought); Knowledge Augmentation (knowledge snippets); Creative Gen. (specification Gen.); Contextualization & Personalization (requirement inversion)
P32	Reasoning & Step-wise Thinking (CoT); Knowledge Augmentation (model text injected)
P33	Reasoning & Step-wise Thinking (K-shot); Contextualization & Personalization (role persona)
P34	Reasoning & Step-wise Thinking (CoT); Meta-cognition & Self-reflection (ask model to justify in 10 words)
P35	Reasoning & Step-wise Thinking (CoT); Meta-cognition & Self-reflection (refine phase); Creative Gen. (new user stories)

TABLE VIII
CATEGORIZATION OF PROMPTING METHODS IN SELECTED STUDIES

PE Categories	PIDs	Num.
Reasoning & Step-wise Thinking	P1, P5, P6, P10, P12, P15, P16, P17, P18, P20, P21, P22, P24, P25, P26, P31, P32, P33, P34, P35	20
Contextualization & Personalization	P1, P3, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P16, P17, P18, P19, P20, P21, P23, P24, P25, P28, P29, P30, P31, P33	26
Info. Extraction & Classification	P2, P4, P11, P12, P15, P20, P23, P24, P27, P29, P30	11
Knowledge Augmentation	P3, P7, P11, P17, P20, P23, P31, P32	8
Meta-cognition & Self-reflection	P3, P10, P13, P21, P34, P35	6
Code Generation	P10, P17, P21, P27	4
Creative Generation	P11, P13, P14, P18, P19, P22, P25, P26, P27, P28, P29, P30, P31, P35	14
Multimodal Understanding		0
Defeater Identification	P9	1
Defect Correction	P11	1

predominantly concentrate on code-based or textual tasks. As an important part of requirements engineering, model-based representation is not frequently explored in PE4RE research, which should be investigated in the future.

- 2) *Lack of investigation in requirements elicitation.* We found that in the selected studies, none of the papers formally reports requirements generation or elicitation. Only four papers (P15, P27, P25, P33) prompt LLMs to ask questions, conduct interviews, or simulate stakeholder interactions. The remaining studies primarily focused on later stages, assuming that requirements were exist.
- 3) *Lack of Systematic Evaluation.* Only 2 (P1, P31) of the 20 Reasoning & Step-wise papers conducted ablation studies. As for Knowledge Augmentation, only eight papers use RAG. In P3 and P20, adding one retrieved chunk improves coverage and SUS scores; however, P7 shows that injecting more than one chunk lowers BLEU/ROUGE. We think the evaluation of the PE4RE paper cannot show a standard evaluation process.

These limitations reveal a maturity gap: while advanced NLP research explores retrieval, self-reflection, vision-language fusion, and cost governance, most PE4RE work still centres on single-prompt templates evaluated on edge cases. The result is promising prototypes that remain difficult to replicate, audit, or certify in industrial RE workflows.

RQ2-2. Road-map and Practical Guidelines for PE4RE. Building on the gaps highlighted in RQ2-1, we propose a four-stage roadmap that can progressively advance PE4RE from prototype studies to an industry-ready discipline.

R1. Multimodal Elicitation & Consistency Checking (*addresses Limitation L1: “text-only focus”*) Tools such as GPT-4o, Gemini 2.5 Pro, and LLaVA-Next now accept images and structured graphics. A promising research direction is to combine UML, BPMN, or SysML diagrams with textual requirements, allowing the model to identify inconsistencies or generate trace links without any loss of information during serialization.

- *Research task:* Fuse textual prompts with visual artefacts,

e.g. UML, wire-frames, or SysML block diagrams to enable cross-modal traceability and inconsistency detection.

- *Starter experiment:* Re-implement P30’s sequence-diagram generator using GPT-4o or Gemini 2.5 Pro with native SVG/XMI input; measure precision and recall of link predictions against a gold standard.
- *Deliverable:* A public benchmark of 100+ multimodal RE cases with labelled links (ground truth).

R2. Elicitation-Phase Prompt Patterns (*addresses Limitation L2: “elicitation under-explored”*) Requirements elicitation is a crucial stage in the RE process. Even though this stage is not thoroughly addressed in LLM-based automation, some RAG methods or knowledge augmentation methods can significantly facilitate the requirements generation.

- Combine *agent templates* (P16) with multi-step *reasoning cues* (P1) to create conversational agents that interview stakeholders, negotiate priorities, and output user stories.
- Evaluate with think-aloud studies: 10 analysts $\times$ 3 projects; measure time-to-first-draft, and requirement completeness level F_1 .
- Curate a library of reusable prompts (e.g. NFR checklist, risk-oriented probes) to standardise elicitation quality.

R3. End-to-End Traceability and Impact Analysis (*addresses Limitation L3: “lack of systematic evaluation”*)

- Retrofit retrieval pipelines (P3, P20) so that each prompt embeds a unique requirement ID; any model-generated change triggers an automatic “blast-radius” report identifying affected tests, designs, and code.
- Define an evaluation protocol: given N changed requirements, measure (i) recall of impacted artefacts, (ii) false-positive rate, and (iii) review effort (min).
- Release evaluation scripts and Docker images to encourage replication studies.

R4. Community Benchmark Suite & Reporting Checklist (*cross-cuts all limitations*)

- Aggregate datasets from R1–R3 plus existing corpora (P1, P26, P30) into a single benchmark covering *text*, *code*, and *diagram* tasks.
- Publish a PE4RE reporting checklist (e.g., model, prompt template, context length, model params, and metrics).
- Host an annual shared task (e.g. at RE ’26) to drive comparable results and foster tooling ecosystems.

Expected Pay-offs. R1–R4 create a virtuous cycle: richer multimodal datasets (R1) enable smarter elicitation agents (R2); explicit IDs and retrieval hooks (R3) support reliable, auditable pipelines, while a benchmark suite (R4) grounds future work in rigorous, replicable evaluation. Collectively, these milestones move PE4RE research from isolated proofs-of-concept to a *standardised, stakeholder-centred, and evidence-based practice* in RE.

VI. THREATS TO VALIDITY

Internal Validity. The first author designed the SLR protocol, which was collaboratively reviewed and refined by all co-authors. Search strings were iteratively adjusted and

customized for each database to optimize retrieval. Study selection followed a three-stage filtering process: (1) title and abstract screening, (2) brief reading with keyword matching, and (3) comprehensive paper review. All authors validated the final selection to ensure robustness. Data extraction was conducted using Google Forms, with all authors participating in pilot testing to ensure consistency. **Construct Validity.** We mitigated threats through automated and manual searches across six scientific databases. All authors refined inclusion/exclusion criteria through extensive discussions. The first and second authors carefully reviewed studies with vague methodologies to reach consensus on inclusion. **Conclusion Validity.** Threats were minimized through validated search and data extraction processes. The data extraction form was designed based on predefined RQs and verified by co-authors before full implementation. Multiple discussions during analysis ensured robust categorization and meaningful conclusions. **External Validity.** We employed comprehensive search strategies following established guidelines. Our focus on peer-reviewed English studies excluded grey literature and opinion pieces to minimize bias, though this may limit some relevant works. The broad temporal scope (2018-present) ensured comprehensive coverage of PE4RE.

VII. CONCLUSION

This work presents the first systematic review of Prompt Engineering for Requirements Engineering (PE4RE), analyzing 35 primary studies from 2018-2025. We propose novel categorizations of PE techniques from technique-oriented and task-oriented perspectives, revealing current trends in target tasks, LLMs employed, and prompting methods. Our analysis identifies critical research gaps and establishes a roadmap for future PE4RE development. Our guidelines aim to advance PE4RE from random prototypes to comprehensive, industry-standard solutions by consolidating current practices and outlining future directions. We anticipate follow-up work to empirically validate these recommendations, create open benchmark datasets, and incorporate PE4RE tools into mainstream RE toolchains. This approach will help accelerate the adoption of LLM-driven methods in real-world SE scenarios.

REFERENCES

- [1] H. F. Hofmann and F. Lehner, "Requirements engineering as a success factor in software projects," *IEEE software*, vol. 18, no. 4, 2001.
- [2] C. Arora, J. Grundy, and M. Abdelrazek, "Advancing requirements engineering through generative ai: Assessing the role of llms," *arXiv preprint arXiv:2310.13976*, 2023.
- [3] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *International conference on data intelligence and cognitive informatics*. Springer, 2023.
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.
- [5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, 2022.
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, 2020.
- [7] B. Kitchenham, L. Madeyski, and D. Budgen, "Segress: Software engineering guidelines for reporting secondary studies," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, 2022.
- [8] K. Petersen, S. Vakkalanka, and L. Kuzniarz, "Guidelines for conducting systematic mapping studies in software engineering: An update," *Information and software technology*, vol. 64, 2015.
- [9] I. Sommerville, R. Lock, and T. Storer, "Information requirements for enterprise systems," *ArXiv*, vol. abs/1209.5246, 2012.
- [10] A. Knutas, Z. Pourzolfaghar, and M. Helfert, "A meta-level design science process for integrating stakeholder needs - demonstrated for smart city services," in *International Conference on Computer-Human Interaction Research and Applications*, 2017.
- [11] I. C. Udousoro, "Effective requirement engineering process model in software engineering," *Software Engineering*, 2020.
- [12] S. Besrou, L. B. A. Rahim, and P. D. D. Dominic, "Exploratory study to assess and evaluate requirement specification techniques using analysis determination requirements framework," *Research Journal of Applied Sciences, Engineering and Technology*, vol. 9, 2015.
- [13] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *arXiv preprint arXiv:2107.13586*, 2021.
- [14] Z. Jiang, Q. Tang, and Z. Wang, "Generative reliability-based design optimization using in-context learning capabilities of large language models," *ArXiv*, vol. abs/2503.22401, 2025.
- [15] S. Minaee, T. Mikolov, N. Nikzad, M. A. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large language models: A survey," *ArXiv*, vol. abs/2402.06196, 2024.
- [16] H. Wan, J. Zhang, Y. Chen, W. Xu, and F. Feng, "Generative ai application for building industry," *ArXiv*, vol. abs/2410.01098, 2024.
- [17] J. D. Velásquez-Henao, C. J. Franco-Cardona, and L. Cadavid-Higuaita, "Prompt engineering: a methodology for optimizing interactions with ai-language models in the field of engineering," *DYNA*, 2023.
- [18] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," *arXiv preprint arXiv:2402.07927*, 2024.
- [19] O. Fagbohun, R. M. Harrison, and A. Dereventsov, "An empirical categorization of prompting techniques for large language models: A practitioner's guide," *arXiv preprint arXiv:2402.14837*, 2024.
- [20] C. Arora, T. Herda, and V. Himm, "Generating test scenarios from nl requirements using retrieval-augmented llms: An industrial study," in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 2024.
- [21] O. Rubin, J. Herzig, and J. Berant, "Learning to retrieve prompts for in-context learning," *arXiv preprint arXiv:2112.08633*, 2021.
- [22] C. Li, J. Wang, Y. Zhang, K. Zhu, W. Hou, J. Lian, F. Luo, Q. Yang, and X. Xie, "Large language models understand and can be enhanced by emotional stimuli," *arXiv preprint arXiv:2307.11760*, 2023.
- [23] Z. Zhang, A. Zhang, M. Li, and A. Smola, "Automatic chain of thought prompting in large language models," *preprint arXiv:2210.03493*, 2022.
- [24] X. Zhao, M. Li, W. Lu, C. Weber, J. H. Lee, K. Chu, and S. Wermter, "Enhancing zero-shot chain-of-thought reasoning in large language models through logic," *arXiv preprint arXiv:2309.13339*, 2023.
- [25] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in neural information processing systems*, vol. 36, 2023.
- [26] Y. Yao, Z. Li, and H. Zhao, "Beyond chain-of-thought, effective graph-of-thought reasoning in language models," *arXiv preprint arXiv:2305.16582*, 2023.
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [28] X. Li, R. Zhao, Y. K. Chia, B. Ding, S. Joty, S. Poria, and L. Bing, "Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources," *arXiv preprint arXiv:2305.13269*, 2023.
- [29] J. Li, D. Li, S. Savarese, and S. Hoi, "Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," *arXiv preprint arXiv:2301.12597*, 2023.
- [30] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds *et al.*, "Flamingo: a visual language model for few-shot learning," *Advances in neural information processing systems*, vol. 35, 2022.

- [31] S. Huang, L. Dong, W. Wang, Y. Hao, S. Singhal, S. Ma, T. Lv, L. Cui, O. K. Mohammed, B. Patra *et al.*, “Language is not all you need: Aligning perception with language models,” *arXiv preprint arXiv:2302.14045*, 2023.
- [32] Z. Peng, W. Wang, L. Dong, Y. Hao, S. Huang, S. Ma, and F. Wei, “Kosmos-2: Grounding multimodal large language models to the world,” *arXiv preprint arXiv:2306.14824*, 2023.
- [33] W. Wu, H. Yao, M. Zhang, Y. Song, W. Ouyang, and J. Wang, “Gpt4vis: what can gpt-4 do for zero-shot visual recognition?” *arXiv preprint arXiv:2311.15732*, 2023.
- [34] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Re-flexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [35] Y. Wang and Y. Zhao, “Metacognitive prompting improves understanding in large language models,” *arXiv preprint arXiv:2308.05342*, 2023.
- [36] H. Gao, T.-E. Lin, H. Li, M. Yang, Y. Wu, W. Ma, and Y. Li, “Self-explanation prompting improves dialogue understanding in large language models,” *arXiv preprint arXiv:2309.12940*, 2023.
- [37] H. S. Zheng, S. Mishra, X. Chen, H.-T. Cheng, E. H. Chi, Q. V. Le, and D. Zhou, “Take a step back: Evoking reasoning via abstraction in large language models,” *arXiv preprint arXiv:2310.06117*, 2023.
- [38] S. A. Ehikioya and E. Guillemot, “A critical assessment of the design issues in e-commerce systems development,” *Eng. Reports*, vol. 2, 2020.
- [39] F. D. Giraldo, S. España, and Ó. Pastor, “Evidences of the mismatch between industry and academy on modelling language quality evaluation,” *ArXiv*, vol. abs/1606.02025, 2016.
- [P14] Ramasamy, Vijayalakshmi; Ramamoorthy, Suganya; Walia, Gursimran Singh; Kulpinski, Eli; Antreasian, Aaron. Enhancing User Story Generation in Agile Software Development Through OpenAI and Prompt Engineering. In *2024 IEEE Frontiers in Education Conference (FIE)*, 2024, pp. 1–8.
- [P15] Arulmohan, Sathurshan; Meurs, Marie-Jean; Mosser, Sébastien. Extracting Domain Models from Textual Requirements in the Era of Large Language Models. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2023, pp. 580–587.
- [P16] Sami, Malik Abdul; Waseem, Muhammad; Zhang, Zheyang; Rasheed, Zeeshan; Systä, Kari; Abrahamsson, Pekka. Early Results of an AI Multi-Agent System for Requirements Elicitation and Analysis. In *Product-Focused Software Process Improvement*, Springer Nature Switzerland, 2025, pp. 307–316.
- [P17] Zhao, Zelong; Zhang, Nan; Yu, Bin; Duan, Zhenhua. Generating Java Code Pairing with ChatGPT. *Theoretical Computer Science*, vol. 1021, Art. 114879, 2024.
- [P18] Görer, Binnur; Aydemir, Fatma Başak. Generating Requirements Elicitation Interview Scripts with Large Language Models. In *Proceedings of the 31st IEEE International Requirements Engineering Conference Workshops (REW)*, IEEE, 2023, pp. 44–51.
- [P19] Oswal, Jay U.; Kanakia, Harshil T.; Suktel, Devvrat. Transforming Software Requirements into User Stories with GPT-3.5 – An AI-Powered Approach. In *2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, IEEE, 2024, pp. 913–920.
- [P20] Kotama, I. Nyoman Darma; Funabiki, Nobuo; Panduman, Yohanes Yohanie Fridelin; Brata, Komang Candra; Pradhana, Anak Agung Surya; Desnanjaya, I. Gusti Made Ngurah. Implementation of Sensor Input Setup Assistance Service Using Generative AI for SEMAR IoT Application Server Platform. *INFORMATION*, vol. 16, no. 2, Feb. 2025.
- [P21] Wang, Yizhuo; Cui, Shiqi; Wan, Rongxin; Wang, Jingyi; Wang, Fanggang. Large Language Models Based Communication Simulation Platform. In *2024 IEEE 24th International Conference on Communication Technology (ICCT)*, IEEE, 2024, pp. 1891–1895.
- [P22] Blasek, Nico; Eichenmüller, Karl; Ernst, Bastian; Götz, Niklas; Nast, Benjamin; Sandkuhl, Kurt. Large Language Models in Requirements Engineering for Digital Twins. In *CEUR Workshop Proceedings*, vol. 3645, 2023.
- [P23] An, Zhiqun; Wan, Hongyan; Xiong, Teng; Wang, Bangchao. LePB-SA4RE: A Lexicon-Enhanced and Prompt-Tuning BERT Model for Evolving Requirements Elicitation from App Reviews. *Applied Sciences – Basel*, vol. 15, no. 5, 2025.
- [P24] Fazelnia, Mohamad; Koscinski, Viktorja; Herzog, Spencer; Mirakhorli, Mehdi. Lessons from the Use of Natural Language Inference (NLI) in Requirements Engineering Tasks. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, IEEE, 2024, pp. 103–115.
- [P25] Pereira, Juanan; López, Juan-Miguel; Garmendia, Xabier; Azanza, Maider. Leveraging Open Source LLMs for Software Engineering Education and Training. In *2024 36th International Conference on Software Engineering Education and Training (CSEE&T)*, IEEE, 2024, pp. 1–10.
- [P26] Ferrari, Alessio; Abualhaija, Sallam; Arora, Chetan. Model Generation with LLMs: From Requirements to UML Sequence Diagrams. In *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*, IEEE, 2024, pp. 291–300.
- [P27] Zhao, Mengyan; Tao, Ran; Huang, Yanhong; Shi, Jianqi; Qin, Shengchao; Yang, Yang. NL2CTL: Automatic Generation of Formal Requirements Specifications via Large Language Models. In *Lecture Notes in Computer Science (LNCS)*, vol. 15394, Springer, 2024, pp. 1–17.
- [P28] Chen, Boqi; Chen, Kua; Hassani, Shabnam; Yang, Yujing; Amyot, Daniel; Lessard, Lysanne; Mussbacher, Gunter; Sabetzadeh, Mehrdad; Varró, Dániel. On the Use of GPT-4 for Creating Goal Models: An Exploratory Study. In *Proceedings of the IEEE 31st International Requirements Engineering Conference Workshops (REW)*, IEEE, 2023, pp. 262–271.
- [P29] Sivakumar, Mithila; Belle, Alvine B.; Shan, Jinjun; Shahandashti, Kimya Khakzad. Prompting GPT-4 to Support Automatic Safety Case Generation. In *Expert Systems with Applications*, vol. 255, Elsevier, 2024, p. 124653.
- [P30] Ruan, Kun; Chen, Xiaohong; Jin, Zhi. Requirements Modeling Aided by ChatGPT: An Experience in Embedded Systems. In *Proceedings of the IEEE 31st International Requirements Engineering Conference Workshops (REW)*, IEEE, 2023, pp. 170–177.
- [P31] Santos, Sarah; Breaux, Travis; Norton, Thomas; Haghighi, Sara; Ghanavati, Sepideh. Requirements Satisfiability with In-Context Learning. In *Proceedings of the IEEE 32nd International Requirements Engineering Conference (RE)*, IEEE, 2024, pp. 168–179.
- [P32] Luitel, Dipeeka; Nejati, Shiva; Sabetzadeh, Mehrdad. Requirements-Driven Slicing of Simulink Models Using LLMs. In *Proceedings of the IEEE 32nd International Requirements Engineering Conference Workshops (REW)*, IEEE, 2024, pp. 72–82.
- [P33] Li, Yishu; Keung, Jacky; Yang, Zhen; Ma, Xiaoxue; Zhang, Jingyu; Liu, Shuo. SimAC: Simulating Agile Collaboration to Generate Acceptance Criteria in User Story Elaboration. *Automated Software Engineering*, vol. 31, no. 2, Nov. 2024.
- [P34] Preda, Anamaria-Roberta; Mayr-Dorn, Christoph; Mashkoor, Atif; Egyed, Alexander. Supporting High-Level to Low-Level Requirements Coverage Reviewing with Large Language Models. In *Proceedings of the IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, IEEE/ACM, 2024, pp. 242–253.
- [P35] Rahman, Tajmilur; Zhu, Yuecai; Maha, Lameya; Roy, Chanchal; Roy, Banani; Schneider, Kevin. Take Loads Off Your Developers: Automated User Story Generation Using Large Language Model. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, 2024.

APPENDIX

- [P1] Siddeshwar, Vaishali; Alwidian, Sanaa; Makrehchi, Masoud. A Comparative Study of Large Language Models for Goal Model Extraction. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ACM, 2024, pp. 253–263.
- [P2] Helmezi, Robert K.; Cevik, Mucahit; Yildirim, Savas. A Prompt-based Few-shot Learning Approach to Software Conflict Detection. In *Proceedings of the 32nd Annual International Conference on Computer Science and Software Engineering*, IBM Corp., 2022, pp. 101–109.
- [P3] Shi, Lu; Qi, Bin; Luo, Jiarui; Zhang, Yang; Liang, Zhanzhao; Gao, Zhaowei; Deng, Wenke; Sun, Lin. Aegis: An Advanced LLM-Based Multi-Agent for Intelligent Functional Safety Engineering. In *EMNLP 2024 – Conference on Empirical Methods in Natural Language Processing (Industry Track)*, 2024, pp. 1571–1583.
- [P4] Alawaji, Batool; Hakami, Mona; Alshemaimri, Bader. Evaluating Generative Language Models with Prompt Engineering for Categorizing User Stories to Its Sector Domains. In *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, IEEE, 2024, pp. 1–8.
- [P5] S, Shakthi; Srivastava, Pratibha; L, Ravi Kumar; Prasad, S.G. Automated Test Case Generation for Satellite FRD Using NLP and Large Language Model. In *2024 4th International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, 2024, pp. 1–8.
- [P6] Sánchez Cuadrado, Jesús; Pérez-Soler, Sara; Guerra, Esther; De Lara, Juan. Automating the Development of Task-oriented LLM-based Chatbots. In *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, ACM, 2024.
- [P7] Arora, Chetan; Herda, Tomas; Homm, Verena. Generating Test Scenarios from NL Requirements Using Retrieval-Augmented LLMs: An Industrial Study. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, IEEE, 2024, pp. 240–251.
- [P8] Seifert, Daniel; Jöckel, Lisa; Trendowicz, Adam; Ciolkowski, Marcus; Honroth, Thorsten; Jedlitschka, Andreas. Can Large Language Models (LLMs) Compete with Human Requirements Reviewers? – Replication of an Inspection Experiment on Requirements Documents. *Lecture Notes in Computer Science*, vol. 15452 LNCS, Springer, 2025, pp. 27–42.
- [P9] Gohar, Usman; Hunter, Michael C.; Lutz, Robyn R.; Cohen, Myra B. CoDefeater: Using LLMs to Find Defeaters in Assurance Cases. In *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024, pp. 2262–2267.
- [P10] Bai, Xingyuan; Huang, Shaobin; Wei, Chi; Wang, Rui. Collaboration Between Intelligent Agents and Large Language Models: A Novel Approach for Enhancing Code Generation Capability. *Expert Systems with Applications*, vol. 269, Art. 126357, 2025.
- [P11] Zhang, Li; Ouyang, Liubo; Xu, Zhuoqun. Defect Correction Method for Software Requirements Text Using Large Language Models. In *2024 International Joint Conference on Neural Networks (IJCNN)*, 2024, pp. 1–8.
- [P12] Bragilovski, Maxim; Van Can, Ashley T.; Dalpiaz, Fabiano; Sturm, Arnon. Deriving Domain Models from User Stories: Human vs. Machines. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, 2024, pp. 31–42.
- [P13] De Vito, Gabriele; Palomba, Fabio; Gravino, Carmine; Di Martino, Sergio; Ferrucci, Filomena. ECHO: An Approach to Enhance Use Case Quality Exploiting Large Language Models. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2023, pp. 53–60.